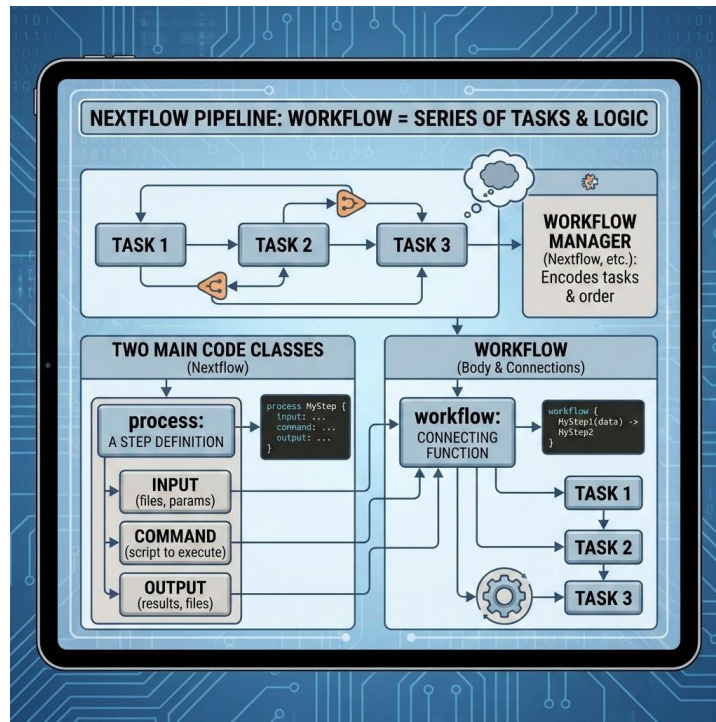


Hello Nextflow

Code introduction: workflow & process

Key components of a Nextflow pipeline

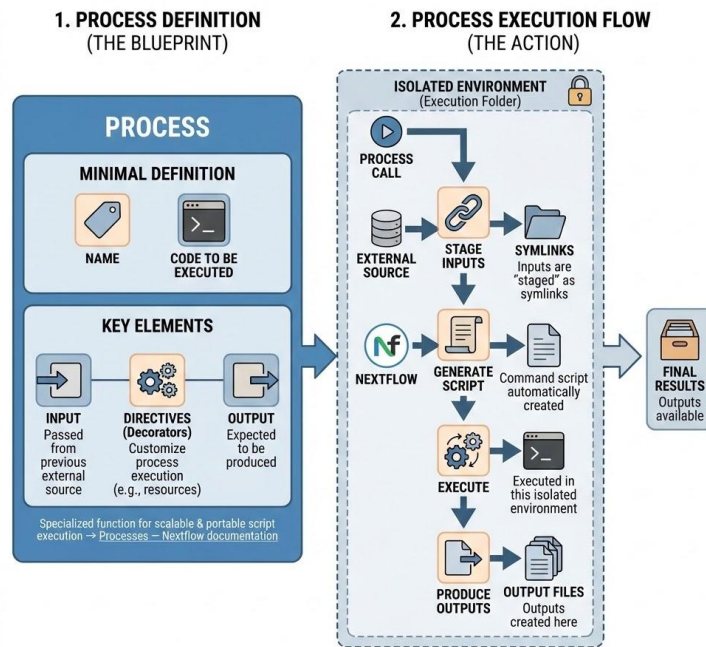
- A workflow is a series of “tasks” to be carried out in a specific order and with pre-defined logics
- Every workflow manager (Nextflow, SnakeMake etc.) have its own way of encoding the different tasks to be carried out until the end of the analysis
- In Nextflow you have two main code classes to build your very first pipeline
 - **process**: the definition of a step, minimally composed by input, output and command to execute
 - **workflow**: body where to write the functions to connect different processes



Key components and principles of a process

- A **process** is a specialized function for executing scripts in a scalable and portable manner → [Processes — Nextflow documentation](#)
- The very minimal definition include a **name** and a **code to be executed**, but generally it is needed to define other elements:
 - **input**: elements that are passed to the process from a previous external source
 - **output**: elements that are expected to be produced from the process
 - **decorators**: additional objects, in Nextflow are called “directives”, used to customize the process in many different ways
- A each time a **process** is called is executed into an isolated environment
 - Inputs are “staged” into a specific execution folder, generally as symlinks
 - The process command is written in a script automatically created by Nextflow
 - The process is executed in this isolated environment, so the outputs will be created there

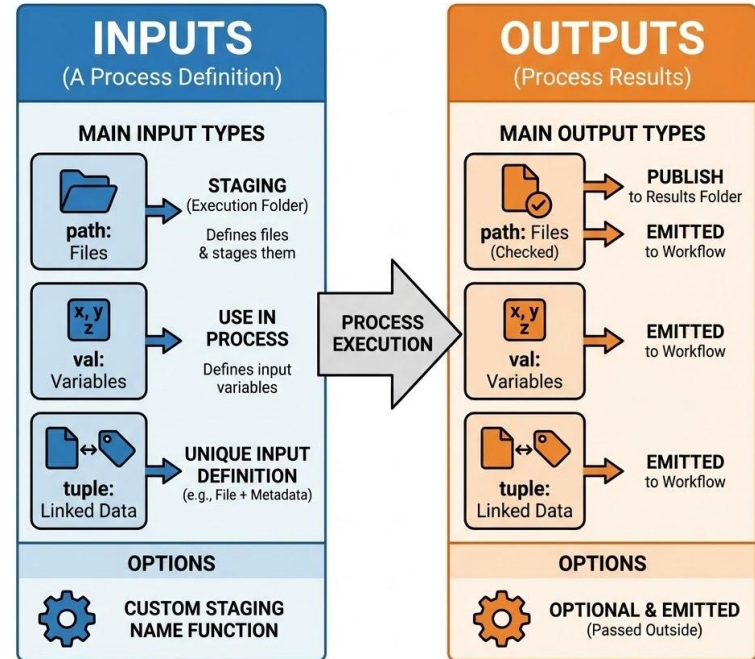
NEXTFLOW PROCESS: DEFINITION & EXECUTION FLOW



Input & output process definitions

- A **process** can have a number (as much as you like) number of inputs that can be defined in many different ways; the most important are:
 - **path**: used to define input files and staging them correctly in the in execution folder
 - **val**: used to define input variables and use them in your process
 - **tuple**: used to put together multiple of the previous in a unique input definition → for example to link together files and metadata associated to that file
 - Full list here → [Processes — Nextflow documentation](#)
- As for the inputs, the same apply to outputs → you can define as much outputs as you want as **path**, **val** or **tuple** ()
 - The existence of output files (e.g. the one defined by **path**) are generally checked every time a given process is executed
 - Defined output files can be published them to the results folder
 - Full list here → [Processes — Nextflow documentation](#)
- Each input/output definition may have a series of options related to them
 - Input files (e.g. when you declare them with **path**) can have a custom staging name function
 - Output files can be defined as optional and/or be “emitted” as previously so that they can be passed outside the process to your workflow

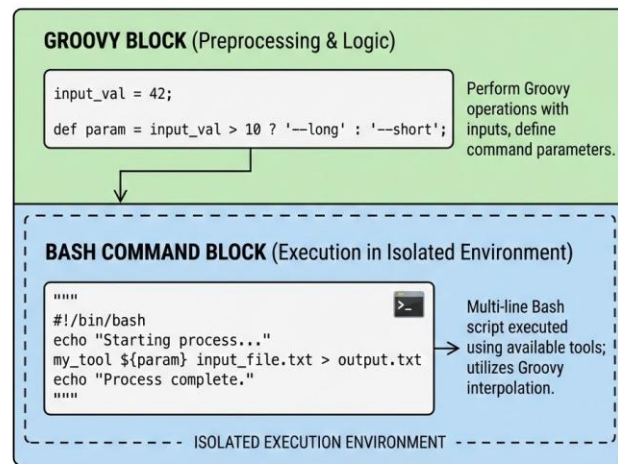
NEXTFLOW PROCESS INPUTS & OUTPUTS



Command definition: the “script” section

- In the `script` section you can define, as a string, the Bash command to execute for a given `process` → [Processes — Nextflow documentation](#)
- The command is executed in the **isolated environment**, so it will use the commands and tools available in that given environment
- This string can be also multi-line, so that you can **write it down as any Bash** script you would normally write
- This string can be **preceded by any Groovy statement** where you can perform any Groovy operation with the defined input → for example you can define specific command parameters given specific values passed to `val` and pass them to the command string using interpolation

NEXTFLOW PROCESS: SCRIPT SECTION

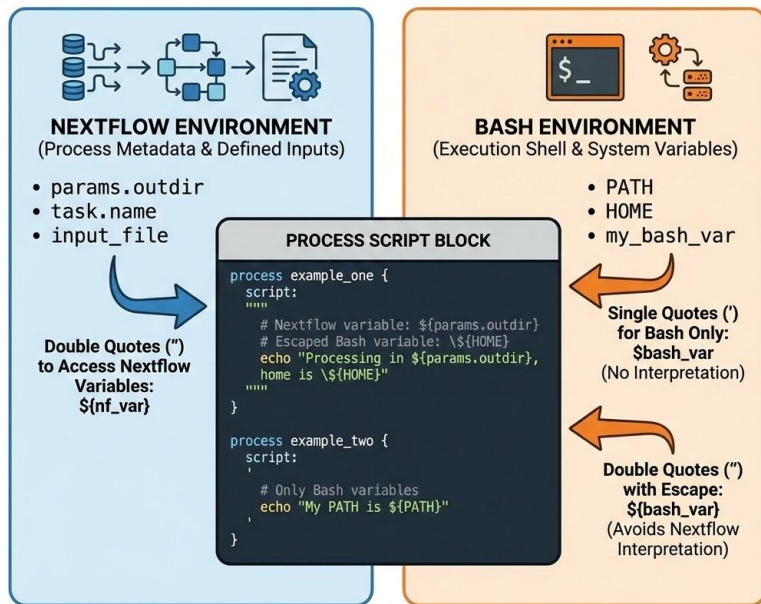


The script section defines the Bash command as a string, preceded by optional Groovy statements for dynamic command generation.

Nextflow and Bash variables in the process

- When writing a process, remember that you have “two environments” in the same place instance with their own environmental variables:
 - Nextflow: defined input, process metadata (for example the process name)
 - Bash: anything you would define during the process or general variables such as `${PATH}`
- If you don't need any Nextflow variable, you can use single quotes (') to call Bash variables as they are (e.g. `$my_var`)
- If you need to access Nextflow variables, then you need to use double quotes (") and to escape Bash variables (e.g. `\${my_var}`) because the Nextflow variables are called with the usual `$` notation (e.g. `${my_nf_var}`, `\${my_bash_var}`)

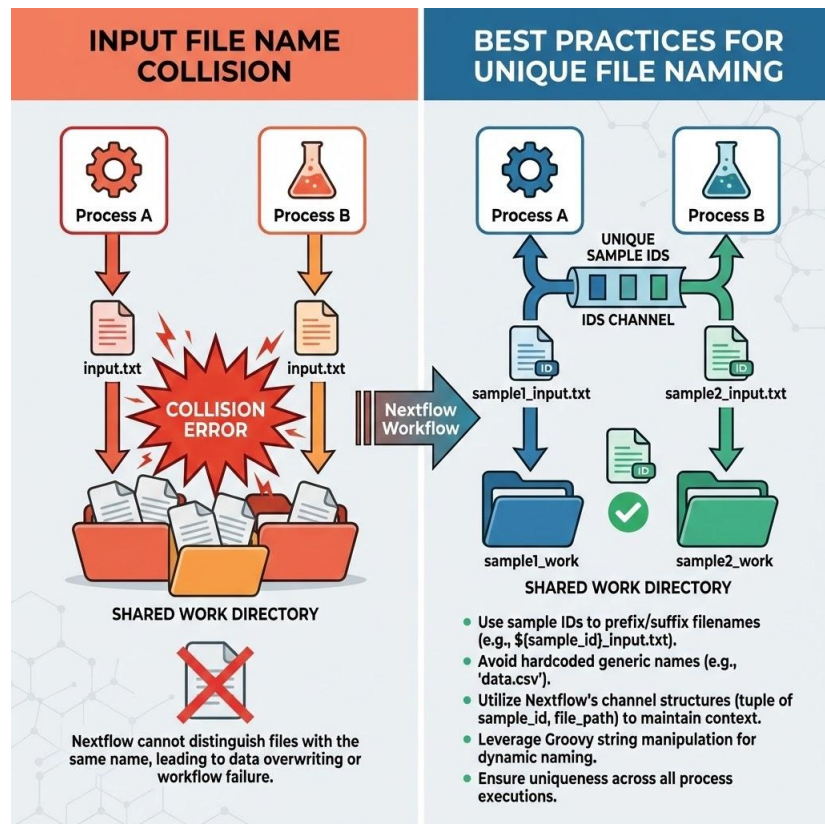
NEXTFLOW & BASH VARIABLES IN NEXTFLOW SCRIPTS



Use **double quotes (")** for Nextflow variables and to escape Bash variables.
Use **single quotes (')** for only Bash variables to avoid conflict.

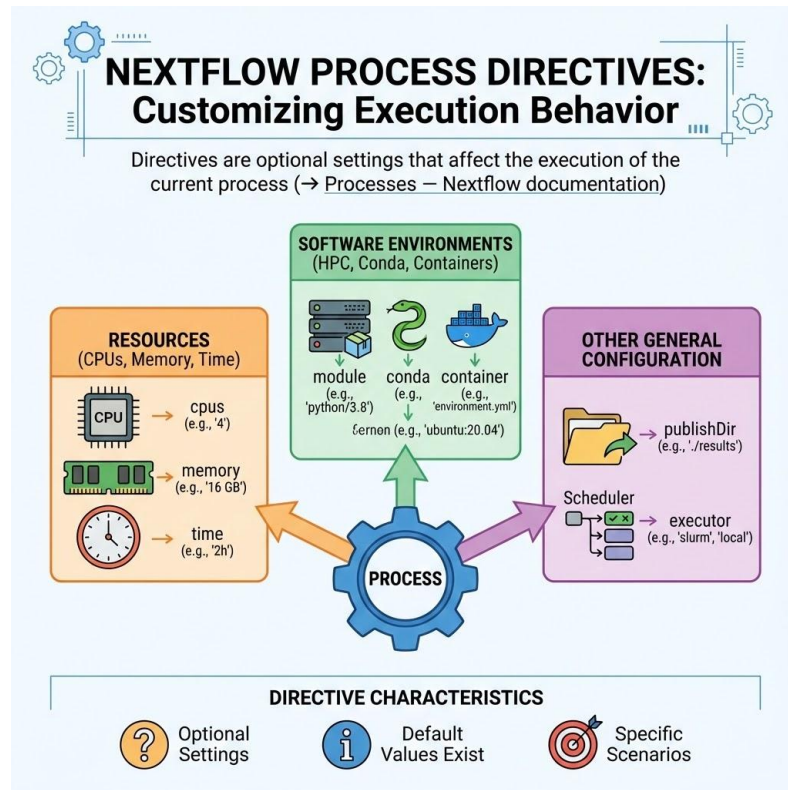
Input file name collision

- Whenever you have a job that takes multiple files as input, each one of these must have a unique name
 - Since every task is executed in an isolated environment and since the input files are staged TOGETHER in a single folder, if you have files that have the same name Nextflow will fail
- To avoid this error, find a “unique ID” to name your time for each task
 - For example, if you need to execute a task **by sample**, you could name your file `${sample}.txt`
 - You can apply this reasoning for every combination of variables that uniquely define a task execution
 - If you have a task that you know it will be executed only once, you don't need to worry about this and you can use a “flat” name for your file
- To name your files, you can use metadata that you bring together with a file in your channel and access that metadata as a Nextflow variable in the `script` section



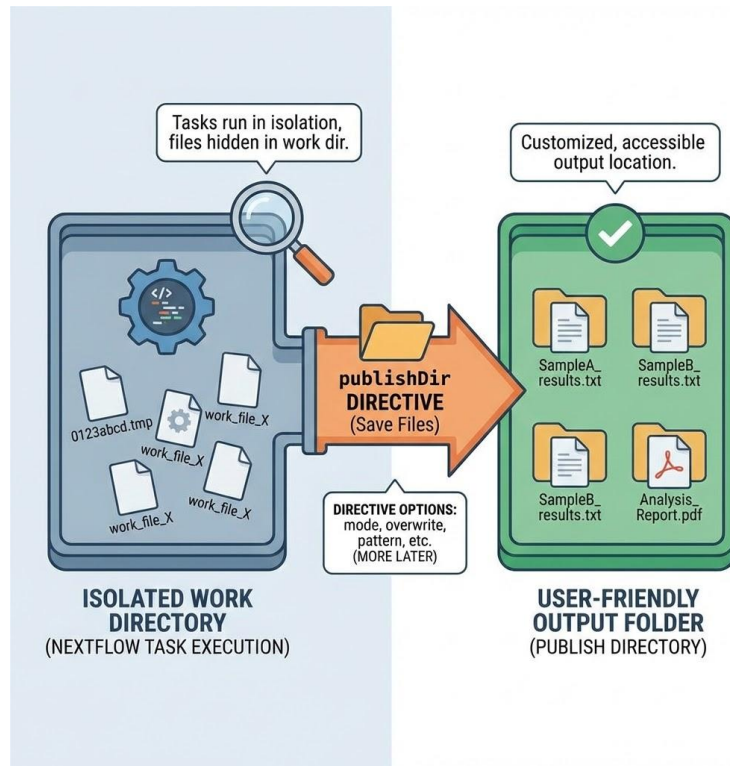
Decorators for your processes: directives

- Directives are optional settings that affect the execution of the current process → [Processes — Nextflow documentation](#)
- There are A LOT of different directives; some will have default values even if you don't declare them while others are specific for given situation and will not be used in specific scenarios
- Examples of executors can be related to:
 - Resources: CPUs, Memory or time to assign to each task execution
 - Software environments: HPC modules, conda environments, containers
 - Other general configuration
 - Publish files to a given output folder
 - Executor → which scheduler to use
- Comprehensive documentation of all the directives
→ [Process reference — Nextflow documentation](#)



publishDir directive for publishing files to your output folder

- As we said, tasks are executed in an isolated context and directory in the work space
- If we want to save files from that isolated space to another location, we need to use the `publishDir` directive
- With this we can define the output folder and customize where to put all the output files that we need in a user friendly manner, without looking for them in the work dir
- The directive have multiple options associated to the path where to publish the files; one of these is `mode`
 - With `mode` you can control **how** a file is published from the work to the output directive
 - By default `mode` is symlink, so you will not copy the files to your output dir but you will symlink them
 - Symlinks allow you to save space but once you clean up your work folder your symlink will be broken
 - Copying files will temporarily occupy more space until you don't clean up your work dir
- `publishDir` complete reference: [Process reference — Nextflow documentation](#)
- **IMPORTANT:** the copy of the files to the destination dir is asynchronous, meaning that the copy of the files doesn't necessarily start when the output file is created
 - There may be an important number of files to copy, Nextflow will create a "queue to copy"
 - **NEVER** use the published files in your workflow → the copy can take time, the files will be corrupted during the copy



Recap and key points

- Nextflow pipelines are composed by workflows
- In workflows you can define the different steps using processes
 - The process is defined outside the workflow and then called inside the workflow
- Processes are generally composed by different elements
 - Inputs
 - Outputs
 - Command to be executed (script)
 - Directives → decorators that can be used to customize the process

